

Programação Web

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Este guia acompanha os diapositivos teóricos sobre Páginas Web Dinâmicas. Irá construir uma aplicação web completa do zero, começando com um perfil estático e evoluindo-o para um sistema dinâmico com autenticação de utilizadores, mapas em tempo real e suporte por chat.

Tecnologias utilizadas:

- **Frontend:** HTML5, CSS3 (Tema Nord Light), Vanilla JavaScript.
- **Backend 1:** Node.js (Express) para Autenticação e Chat.
- **Backend 2:** Python (FastAPI) para Geolocalização e Processamento de Dados.
- **Infraestrutura:** Docker & Docker Compose.

Fase 1: Configuração do Projeto e Estrutura Estática

Passo 0: Instalação e Verificação

Antes de escrever código, certifique-se de que o seu ambiente está pronto.

1. **Abra o seu terminal.**
2. **Verifique o Docker:** `docker --version` e `docker compose version`
3. **Verifique o Node.js (Opcional):** `node -v`

Passo 1: Estrutura de Pastas

1. Crie uma pasta principal chamada `ex10`.
2. Dentro dela, crie três subpastas: `frontend`, `auth-service` e `geo-service`.
3. Crie um ficheiro `compose.yml` na raiz.

Passo 2: O Docker Compose Base

Abra o `compose.yml` e cole este código:

```
services:
  # 1. Servidor Frontend (Nginx)
  web:
    image: nginx:alpine
    container_name: frontend_server
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html
```

Passo 3: O Perfil Estático (HTML)

Abra o `frontend/index.html`. Note a `div chatWidget` no fundo; esta será utilizada no Passo 11.

```
<!DOCTYPE html>
<html lang="pt">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>0 Meu Perfil Dinâmico</title>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <header>
    <h1>Perfil de Utilizador</h1>
    <nav>
      <button id="loginBtn">Login</button>
      <button id="logoutBtn" style="display:none;">Logout</button>
    </nav>
  </header>

  <main id="app">
    <dialog id="loginDialog">
      <form id="loginForm">
        <h2>Bem-vindo de volta</h2>
        <input type="text" id="username" placeholder="Nome de utilizador" required>
        <input type="password" id="password" placeholder="Palavra-passe" required>
        <button type="submit">Entrar</button>
        <button type="button" id="cancelLogin">Cancelar</button>
      </form>
    </dialog>

    <div id="contentArea" class="hidden">
      <section class="card profile-card">
        
        <h2 id="welcomeMsg">Olá, Utilizador</h2>
        <p>Estudante Full Stack</p>
      </section>

      <section class="card gallery-card">
        <h3>Galeria de Fotos</h3>
        <div id="galleryGrid" class="grid"></div>
      </section>

      <section class="card map-card">
        <h3>Rastreador de Localização em Tempo Real (WebSocket)</h3>
        <div id="map"></div>
      </section>

      <div id="chatWidget">
        <div id="chatHeader">Chat de Suporte</div>
        <div id="chatMessages"></div>
        <input type="text" id="chatInput" placeholder="Escreva uma mensagem...">
      </div>

      <div id="guestMessage">
        <p>Por favor, inicie sessão para ver o painel de controlo.</p>
      </div>
    </main>

    <footer>
      <p>&copy; 2025 Engenharia Web</p>
    </footer>

```

```

    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
</body>
</html>

```

Passo 4: Estilização Nórdica (CSS)

Abra o frontend/style.css. Isto define o layout e a posição da Janela de Chat.

```

:root {
  --polar-night: #2E3440;
  --snow-storm: #ECEFF4;
  --frost-1: #8FBCBB;
  --frost-2: #88C0D0;
  --frost-3: #81A1C1;
  --frost-4: #5E81AC;
  --aurora-red: #BF616A;
}
body {
  font-family: 'Noto Sans', sans-serif;
  background-color: var(--snow-storm);
  color: var(--polar-night);
  margin: 0;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
header {
  background-color: var(--frost-4);
  color: white;
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
button {
  background-color: var(--frost-3);
  color: white;
  border: none;
  padding: 0.5rem 1rem;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
}
button:hover { background-color: var(--frost-2); }
main {
  flex: 1;
  padding: 2rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
}
.card {
  background: white;
  padding: 1.5rem;
  border-radius: 8px;
  margin-bottom: 2rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}
.hidden { display: none; }
dialog {
  border: 1px solid var(--frost-2);

```

```

    border-radius: 8px;
    padding: 2rem;
}
dialog::backdrop { background: rgba(46, 52, 64, 0.5); }

/* Estilização do Widget de Chat */
#chatWidget {
    position: fixed;
    bottom: 20px;
    right: 20px;
    width: 300px;
    background: white;
    border: 1px solid var(--frost-3);
    border-radius: 8px;
    overflow: hidden;
    display: flex;
    flex-direction: column;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
#chatHeader {
    background: var(--frost-4);
    color: white;
    padding: 10px;
    font-weight: bold;
}
#chatMessages {
    height: 200px;
    padding: 10px;
    overflow-y: auto;
    font-size: 0.9rem;
    background-color: #fff;
}
#chatInput {
    border: none;
    border-top: 1px solid #eee;
    padding: 10px;
    outline: none;
}

/* Mapa e Galeria */
#map { height: 300px; width: 100%; border-radius: 4px; }
.grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 1rem;
}
.grid img { width: 100%; border-radius: 4px; }

```

Passo 5: JavaScript Básico

Crie o ficheiro frontend/app.js.

```

console.log("Aplicação Carregada");

const loginBtn = document.getElementById('loginBtn');
const loginDialog = document.getElementById('loginDialog');
const cancelLogin = document.getElementById('cancelLogin');

loginBtn.addEventListener('click', () => loginDialog.showModal());
cancelLogin.addEventListener('click', () => loginDialog.close());

```

Teste da Fase 1: Execute `docker compose up -d` e visite <http://localhost:8080>.

Fase 2: Backend Node.js (Autenticação e Chat)

Passo 6: Configuração do Serviço Node

1. Vá para auth-service/.
2. Crie o package.json:

```
{
  "name": "auth-service",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.18.2", "cors": "^2.8.5", "ws": "^8.13.0" }
}
```

3. Crie o Dockerfile:

```
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Passo 7: Implementar a Lógica do Servidor

Crie o ficheiro auth-service/server.js.

```
const express = require('express');
const cors = require('cors');
const http = require('http');
const WebSocket = require('ws');

const app = express();
const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

app.use(cors());
app.use(express.json());

// Endpoint de Login
const VALID_USER = { username: "admin", password: "123" };
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (username === VALID_USER.username && password === VALID_USER.password) {
    res.json({ success: true, token: "jwt-123" });
  } else {
    res.status(401).json({ success: false, message: "Credenciais inválidas" });
  }
});

// Lógica de WebSocket do Chat
wss.on('connection', (ws) => {
  ws.send('Suporte: Olá! Como posso ajudar?');
  ws.on('message', (message) => {
    setTimeout(() => {
      ws.send(`Suporte: Recebi a mensagem "${message}"`);
    }, 1000);
  });
});

server.listen(3000, () => console.log('Autenticação/Chat a correr na porta 3000'));
```

Passo 8: Atualizar o JS do Frontend

Modifique o frontend/app.js para gerir o login.

```
// Adicione estas referências
const loginForm = document.getElementById('loginForm');
const contentArea = document.getElementById('contentArea');
const guestMessage = document.getElementById('guestMessage');
const logoutBtn = document.getElementById('logoutBtn');

loginForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;

  try {
    const response = await fetch('http://localhost:3000/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await response.json();

    if (data.success) {
      loginDialog.close();
      handleLoginState(true);
    } else {
      alert(data.message);
    }
  } catch (err) { console.error(err); }
});

function handleLoginState(isLoggedIn) {
  if (isLoggedIn) {
    contentArea.classList.remove('hidden');
    guestMessage.classList.add('hidden');
    loginBtn.style.display = 'none';
    logoutBtn.style.display = 'inline-block';

    // Carregar funcionalidades dinâmicas
    loadGallery();
    initChat();
    initMap();
  } else {
    location.reload();
  }
}

logoutBtn.addEventListener('click', () => handleLoginState(false));
```

Passo 9: Atualizar o Docker Compose

Atualize o docker-compose.yml para incluir o serviço auth.

```
services:
  web:
    # ... (configuração existente)
  auth:
    build: ./auth-service
    container_name: auth_server
    ports:
      - "3000:3000"
```

Fase 3: Funcionalidades (Galeria e Chat)

Passo 10: A Galeria de Fotos (JS)

Adicione isto ao final do frontend/app.js:

```
function loadGallery() {
  const galleryGrid = document.getElementById('galleryGrid');
  const images = [
    'https://picsum.photos/id/101/300/200',
    'https://picsum.photos/id/102/300/200',
    'https://picsum.photos/id/103/300/200',
    'https://picsum.photos/id/104/300/200'
  ];
  galleryGrid.innerHTML = '';
  images.forEach(url => {
    const img = document.createElement('img');
    img.src = url;
    galleryGrid.appendChild(img);
  });
}
```

Passo 11: O Cliente de Chat (WebSocket)

Adicione isto ao final do frontend/app.js. Este código torna funcional a Janela de Chat definida no HTML/CSS.

```
function initChat() {
  const chatInput = document.getElementById('chatInput');
  const chatMessages = document.getElementById('chatMessages');

  // Ligar ao WebSocket do Node.js
  const socket = new WebSocket('ws://localhost:3000');

  socket.addEventListener('message', (event) => {
    addMessage(event.data, 'server');
  });

  chatInput.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') {
      const text = chatInput.value;
      socket.send(text);
      addMessage("Tu: " + text, 'user');
      chatInput.value = '';
    }
  });

  function addMessage(text, sender) {
    const div = document.createElement('div');
    div.innerText = text;
    div.style.textAlign = sender === 'user' ? 'right' : 'left';
    div.style.color = sender === 'user' ? '#5E81AC' : '#BF616A';
    div.style.marginBottom = '5px';
    chatMessages.appendChild(div);
    chatMessages.scrollTop = chatMessages.scrollHeight;
  }
}
```

Fase 4: Backend FastAPI (Geolocalização)

Passo 12: Configuração do Serviço Python

1. Vá para geo-service/.
2. Crie o requirements.txt:

```
fastapi
uvicorn
websockets
```

3. Crie o Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Passo 13: Implementar a Lógica Python

Crie o ficheiro geo-service/main.py.

```
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
import asyncio, random

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"])

@app.websocket("/ws/location")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    lat, lon = 40.64427, -8.64554
    try:
        while True:
            lat += random.uniform(-0.001, 0.001)
            lon += random.uniform(-0.001, 0.001)
            await websocket.send_json({"lat": lat, "lng": lon})
            await asyncio.sleep(2)
    except: print("Desconectado")
```

Passo 14: Atualizar o Docker Compose

Adicione o serviço geo ao docker-compose.yml.

```
services:
  # ... web e auth existentes ...
  geo:
    build: ./geo-service
    container_name: geo_server
    ports:
      - "8000:8000"
```

Passo 15: O Mapa (Leaflet + WebSocket)

Adicione isto ao final do frontend/app.js:

```
function initMap() {
  const map = L.map('map').setView([40.64427, -8.64554], 13);
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '© OpenStreetMap contributors'
```



```

    }).addTo(map);

    const marker = L.marker([40.64427, -8.64554]).addTo(map);

    // Ligar ao WebSocket de Python
    const geoSocket = new WebSocket('ws://localhost:8000/ws/location');

    geoSocket.addEventListener('message', (event) => {
        const data = JSON.parse(event.data);
        const newLatLng = [data.lat, data.lng];
        marker.setLatLng(newLatLng);
        map.panTo(newLatLng);
    });
}

```

Passo Final: Execute `docker compose up -d --build` e aproveite a sua aplicação!

Fase 5 (Opcional)

Implemente duas grandes alterações no código:

1. Para uma abordagem mais realista, o servidor deve comparar hashes de palavras-passe em vez das palavras-passe diretamente. Implemente esta modificação na página web e no servidor de autenticação.
2. Crie múltiplas janelas de chat, atualizando a página web e o servidor para suportar esta funcionalidade.